Container Services

Chapter 3

EJB Environment

- Environment is standardized by EJB specification
  - Can use different host to develop and deliver
- Different hosts may have different performance
  - But will provide a common set of services
  - May have additional services to distinguish from other vendors hosts

Container Services

- Dependency injection
- Concurrency
- Instance pooling/caching
- Transactions
- Security
- Timers
- Naming and object stores
- Interoperability
- Lifecycle callbacks
- Interceptors
- Java Enterprise Platform integration
- Persistence

Dependency Injection (DI)

- Have talked about this a lot already
- Since Java EE 6 as CDI spec
- Idea goes back to early theory about reducing coupling between classes
  - see Martin Fowler's paper for Java discussion and chapter 16 in text
- Encourages coding to interfaces which is a good general practice

Concurrency

- Session beans do not support concurrent access
  - Intended to be used by one client at a time
  - Do not need to be thread-safe
- Message-driven bean instances handle one message at a time
  - Also single threaded
- Persistence beans represent data that is shared and may be accessed concurrently
  - Container handles concurrency
  - Transactions ensure data integrity
- Bean developer does not have to make bean's methods thread-safe
  - Non singleton beans prohibited from creating own threads or using synchronization
- See Chapters 5, 6, and 7 for session bean issues

Instance Pooling/Caching

- Clients never access EJBs directly
  - Servers can pool bean instances in some cases
  - Stateless session beans of the same type are interchangeable
    - No client state
  - Message-driven beans of the same type that subscribe to a given destination can be pooled
- Persistence beans can be cached to reduce DB usage

Transactions

- All enterprise beans understand transactions
  - Defers, commits, rollbacks
- How they are handled depends on annotations or deployment parameters
  - Can specify either bean or container manages its transactions (usually container can do it)
- Programmers need to understand ACID
  - Atomicity, Consistency, Isolation, Durability
- Implemented with Java Transaction Service
  - See chapter 17 for more details

Security

- Based on concept of Principal
  - Represents a person or role
  - Represented by java.security.Principal object
- Container validates client and assigns to role
- Depending on security role, server can restrict ability to invoke methods on EJB
- These are done in annotations or EJB deployment descriptors
  - See chapter 15

Timers

- Sometimes activities have to happen at specific times
  - backups, system updates
  - weekly/monthly reports
  - Holiday flyers
  - Auction timeouts
- Java EE servers allow triggering events in a very flexible time-based manner
  - See chapter 19

Naming and Object Stores

- Distributed objects are nice if we can find them
- Naming services allow a server to bind an object to a name and a client to look up the name
- Jakarta EE uses the Java Naming and Directory Interface (JNDI)
  - General JNDI is a wrapper for naming services, such as DNS
  - JNDI is used in a simple manner to find home objects and other resources
  - The client lookup code is similar using RMI
- See Chapter 16

Interoperability

- Jakarta EE standard allows EJBs to be accessed remotely
  - Supports RMI-IIOP for remote method invocation
  - Supports JAX-WS, JAX-RPC, Web Services for Java EE and Web Services Metadata for the Java Platform
  - Supports JAX-RS for RESTful applications
- Both Java and non-Java technologies

Object Distribution

- As far as remote client concerned, session bean is defined by its remote interface or endpoint interface
- Underlying protocol is transparent to client
  - Can be CORBA IIOP, JRMP, SOAP
- Parameters and return values can be
  - Primitives
  - Serializable objects
  - Array or collection of primitives or serializable objects
  - Remote objects
- Enterprise messaging supports message-driven beans

Activation Mechanism

- Stateful session beans maintain client state between method calls
- Passivation saves the bean's client state, allowing the bean to be removed
- Activation restores the bean state after the bean instance has been created
  - If a bean is not used for a while, and server needs the space, it may be passivated
  - If the client then makes a method call on the bean, it will be activated before the call

Lifecycle Callbacks

- EJB container calls annotated methods to save and restore things the container cannot
  - `@jakarta.ejb.PrePassivate`
    - Immediately before passivating a stateful session bean
    - Bean should clean up and release resources
  - `@jakarta.ejb.PostActivate`
    - Immediately after activating a stateful session bean
    - Bean should initialize resources required
- Not needed for serializable instance fields and container-managed resources
- `@jakarta.annotation.PostConstruct` and `@jakarta.annotation.PreDestroy` discussed earlier

Interceptors

- Can factor out common code required over several methods into a single place
  - Interceptor
  - Applied to call chain without impacting core logic
  - Called *cross-cutting* concerns
- The most common cross-cutting concerns are standard – for example, security
- See Chapter 18

Platform Integration

- EJB ties together may other Java frameworks and APIs
  - Java Transaction Service (JTS)
  - Java Persistence API (JPA)
  - Java Naming and Directory Interface (JNDI)
  - Security Services
  - Web Services
  - Contexts and Dependency Injection (CDI)
- Usually the EJB metadata required to use these is a simplified view of the underlying services

Persistence

- Persistence beans represent the data in an application
- They are indeed persistent
  - The state is saved in a database
- Three choices (including legacy)
  - EJB 2.1 Container-managed means the container handles persistence without programming
    - can specify mapping of EJB properties to database fields
  - EJB 2.1 Bean-managed means the programmer handles persistence in the bean code
  - EJB 3.0+ use Java/Jakarta Persistence specification
    - can specify mapping of Bean properties to database fields in a standard manner (Object-to-relational mapping or ORM)
- We will cover Jakarta Persistence beans

Jakarta Persistence

- Entities are Java classes with annotations
- Bean instances are attached or detached from container management
  - using EntityManager service
- EntityManager provides methods to create, find, query, update, and remove beans
- When detached, bean instances can be sent across the network
  - serving as data transfer objects

Object-to-relational Persistence

- Jakarta Persistence spec covers how to map object fields to a relational database
  - previously this was vendor specific
- For prototyping you can allow the server to do the mapping
- For operational systems, you probably want to do the mapping yourself
  - to control the database design

Simple ORM Example

```
@Entity
@Table(name="CABIN")
public class Cabin implements java.io.Serializable {
    private int id;
    private String name;
    private int deckLevel;

    @Id
    @Column(name="ID")
    public int getId() {return id;}
    public void setId(int pk) {id = pk;}

    @Column(name="NAME")
    public String getName() { return name;}
    public void setName(String str) {name = str;}

    @Column(name="DECK_LEVEL")
    public int getDeckLevel() {return deckLevel;}
    public void setDeckLevel(int level){deckLevel = level;}
}
```

What Does Persistence Do?

- You can map relationships and entities across multiple tables
  - 1-1, 1-n, n-1, n-n unidirectional or bidirectional relationships
  - Entity fields can come from a multi-table join
- Container keeps persistence bean instance consistent with the corresponding database entries
- No need for JDBC calls at all
- See chapters 9-14

Summary

- This is the end of the overview, on to specifics
- Be sure you have read and reread chapter 2 on the different bean types and the overall architecture
  - so you do not get lost in the details
- We let the container know what we want via annotations or xml descriptors
  - usually the lecture will cover the annotations
  - read about both
    - Annotations emphasized, however

Developing Your First EJBs

Chapter 4

Developing Your First Beans

- Primary goal of EJB 3.1+ and Jakarta Persistence specifications:
  - Make it as easy as possible to write and deploy an EJB-based application
- Steps required:
  - Write code and compile it
  - combine the classes into a jar file
  - deploy it to the application server and run

Parts of EJBs

- Bean implementation class (session and message-driven beans)
  - Business logic for session or message-driven bean
  - Annotated with bean type
    - @jakarta.ejb.Stateless
    - @jakarta.ejb.Stateful
    - @jakarta.ejb.MessageDriven
- Bean instance (SB and MDB)
  - Created by container, not client
  - Not exposed to client, abstracted as *client view*
  - (Entity beans *can* be created by client with **new**)

EJB Client View

- Client view (session and message-driven beans)
  - Contract for client to interact with EJB
  - Business, component or endpoint interface for session beans
    - At least one view must be defined
  - No client view for message driven beans
  - (clients interact directly with entity beans)
- EJB Proxy (session beans)
  - What the client calls to interact with EJB
  - Entered into Global JNDI when EJB deployed
    - client can look up reference

Types of Client Views

- Local or Remote (session beans)
  - Refers to relationship between client and JVM running container
  - Client running in same JVM can use local interface
  - Client running in separate JVM uses remote interface
  - Within a single application often use local interfaces
    - Much more efficient than remote interfaces
    - Avoid network stack and use references instead of serializing values passed

Session Bean Interfaces

- Business interface(session beans)
  - An interface annotated with @jakarta.ejb.Local or @jakarta.ejb.Remote
  - Simplified view introduced in EJB 3.0
  - Defines business methods supported by a session bean
- Endpoint interface (session beans) - legacy
  - Defines business methods that can be accessed from outside the EJB container via SOAP
  - Interface annotated with @jakarta.jws.WebService

Message Interface

- Message-driven beans implement the message interface
  - Defines methods by which messaging systems deliver messages to the bean
  - Typically implements jakarta.jms.MessageListener for using the Java Messaging System (JMS)
    - Built into Jakarta EE application servers.

EJB Legacy Interfaces

- Component Interface (legacy session beans)
  - Defined in EJB 2.x specification and later
  - Also called local or remote interfaces
  - Interface which extends jakarta.ejb.EJBObject or jakarta.ejb.EJBLocalObject and contains business methods
  - Obtained from methods of home interface
- Home interface (legacy session beans)
  - Used to create, find, remove session bean references
  - Extends jakarta.ejb.EJBHome or jakarta.ejb.EJBLocalHome

Coding a Session Bean

- Chapter 4 example creates a SLSB to expose a calculator service
  - And how to test it
- First decide what services to provide
  - Write this as an interface
- Might need both a remote and local interface
  - Write the common interface and extend it for the local and remote interfaces

CalculatorCommonBusiness

```
package org.jboss.ejb3.examples.ch04.firstejb;
/**
 * Contains the contract for operations common to
 * all business interfaces of the CalculatorEJB
 * @author <a href="mailto:andrew.rubinger@jboss.org">ALR</a>
 */
public interface CalculatorCommonBusiness {
    /**
     * Adds all arguments
     * @return The sum of all arguments
     */
    int add(int... arguments);
}
```

Local and Remote Interfaces

```
package org.jboss.ejb3.examples.ch04.firstejb;
/** ... javadoc */
public interface CalculatorRemoteBusiness extends
CalculatorCommonBusiness{
}
```

```
package org.jboss.ejb3.examples.ch04.firstejb;
public interface CalculatorLocalBusiness extends
CalculatorCommonBusiness {


}
```

Base Business Class

```
package org.jboss.ejb3.examples.ch04.firstejb;
import org.jboss.logging.Logger;
public class CalculatorBeanBase implements CalculatorCommonBusiness {
   private static final Logger log =
Logger.getLogger(CalculatorBeanBase.class);
   public int add(final int... arguments) {
      final StringBuffer sb = new StringBuffer();
      sb.append("Adding arguments: ");
      int result = 0;
      for (final int arg : arguments) {
         result += arg;
         sb.append(arg);
         sb.append(" ");
      }
      log.info(sb.toString());
      log.info("Result: " + result);
      return result;
   }
}
```

Simple Bean

```
package org.jboss.ejb3.examples.ch04.firstejb;
import jakarta.ejb.Local;
import jakarta.ejb.Stateless;

@Stateless
@Local(CalculatorLocalBusiness.class)
public class SimpleCalculatorBean extends CalculatorBeanBase
                                    implements CalculatorCommonBusiness {
   // Implementation supplied by common base class
}
```

No-Interface Local Session Bean

- Easiest to code

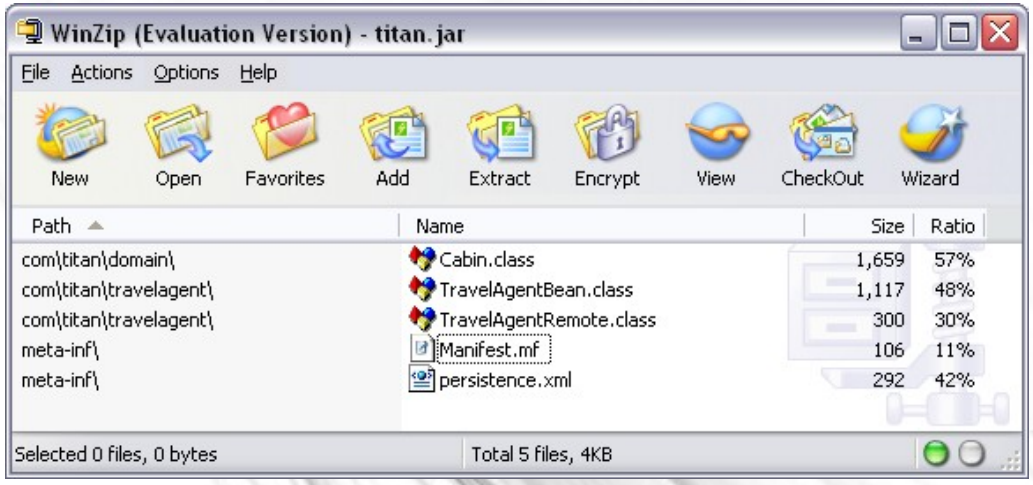
```
package org.jboss.ejb3.examples.ch04.firstejb;

import jakarta.ejb.LocalBean;
import jakarta.ejb.Stateless;
@Stateless
@LocalBean
public class NoInterfaceViewCalculatorBean extends CalculatorBeanBase
{
   // Implementation in base class
}
```

Testing

- See text example for test cases
- Maven setup will automatically run JUnit tests
  - Put the classes in the test directory and put in a dependency for JUnit in the pom
- Integration tests are possible without having to deploy to a server
  - Using Arquillian

Packaging: the jar File

- Some EJB servers may allow beans to be deployed in an unzipped format
- The standard is to package the bean class files and any additional configuration files into a jar file
  - Similar to a war file, but simpler
- Configuration information goes into a META-INF directory



Deploying the EJB jar

- Depending on the application, you may need to create the database and tables
  - See database course for that
  - For prototyping, may use autogeneration
- Deployment is a server-specific process
  - Some, like Wildfly, have a deployment directory
  - Just copy the jar file there
- Maven can build the jar and it can be used as a depencency in other Maven Projects
- Maven war projects as last term still work

Creating a Client Application

- Not as common as browser/mobile clients
- Remote client needs to
  - Find EJB server
  - Locate Remote Interface
  - Interact with EJB to get EJBs and call its methods
- Only tricky bit is locating the server and Remote interface
  - Need a naming service, using JNDI

Finding the Remote Interface: JNDI

- Java Naming and Directory Interface (JNDI) is a wrapper on top of naming services
- EJB servers use it to publicize resources
- If you are running under the EJB server, you automatically know where to find JNDI
- As a client you need to specify where to find it
  - typically in a properties file
  - parameters are server-specific, see server documentation
  - e.g. client-config/jndi.properties for jBoss
- With the properties you get an initial context, under which you can lookup objects by name.

Finding the Remote Object

- Get the initial context
  - `Context jndiContext = new javax.naming.InitialContext();`
- Look up the Remote Object
  - `Object ref = jndiContext.lookup ("ManyViewCalculatorBean/remote");`
- Narrow (EJB 2 beans) and cast it
  - `CalculatorRemoteBusiness dao = (CalculatorRemoteBusiness) PortableRemoteObject.narrow(ref, CalculatorRemoteBusiness.class);`
- Now we can use the object as if it were a normal object with methods

Summary

- Chapter 4 illustrates creating a session bean with unit and integration tests
- The sample code shows how to set up the project with maven and git
  - The resulting jar file can be installed in a Java EE 6+ server (including Jakarta 10+)
- The simple bean and no interface bean are the models for you to use
- Next week we will look at session beans in more detail